

Bug 47071 — FIFO per timecard via outbound integration queue

Work item

- ID: 47071
- Title: Duplicate hours appears when user submits MV timecard on mobile
- Type: Bug
- ADO: https://dev.azure.com/Amergis/MaxView/_workitems/edit/47071
- Parent: 22495
- Branch: `bug/47071-fifo-outbound-queue`

Status: IMPLEMENTED. This document describes the application-level FIFO fix shipped for bug 47071. Serialization is enforced through `IntegrationQueueOutboundTimecards` plus completion callbacks — not Azure Service Bus sessions. A broker-session alternative was investigated earlier and is retained below as a deferred option.

Problem recap

Mobile "add entries then Submit without saving" issues two HTTP calls back-to-back (`POST /api/timecards` then `POST /api/timecards/submit`). Each call runs `IntegrateTimecard` → `QueueTimecardOutbound` → publishes a `TimecardIntegrationExport` message. Because the code runs fast, both messages reached the broker almost simultaneously and the consumer processed them concurrently. The second message inserted a duplicate set of `TimesheetLineEntry` rows in RecruitCare while MaxView's own DB stayed clean.

Root cause: **lack of serialization per timecard**. Nothing guaranteed that integration #2 for a given timecard ran only after integration #1 completed.

The previous `QueueTimecardOutbound` behavior made this worse: it **cancelled** the prior pending queue row and immediately inserted a new one, which did not stop an already-published or in-flight message from running concurrently with the next publish.

Chosen approach — application-level FIFO queue

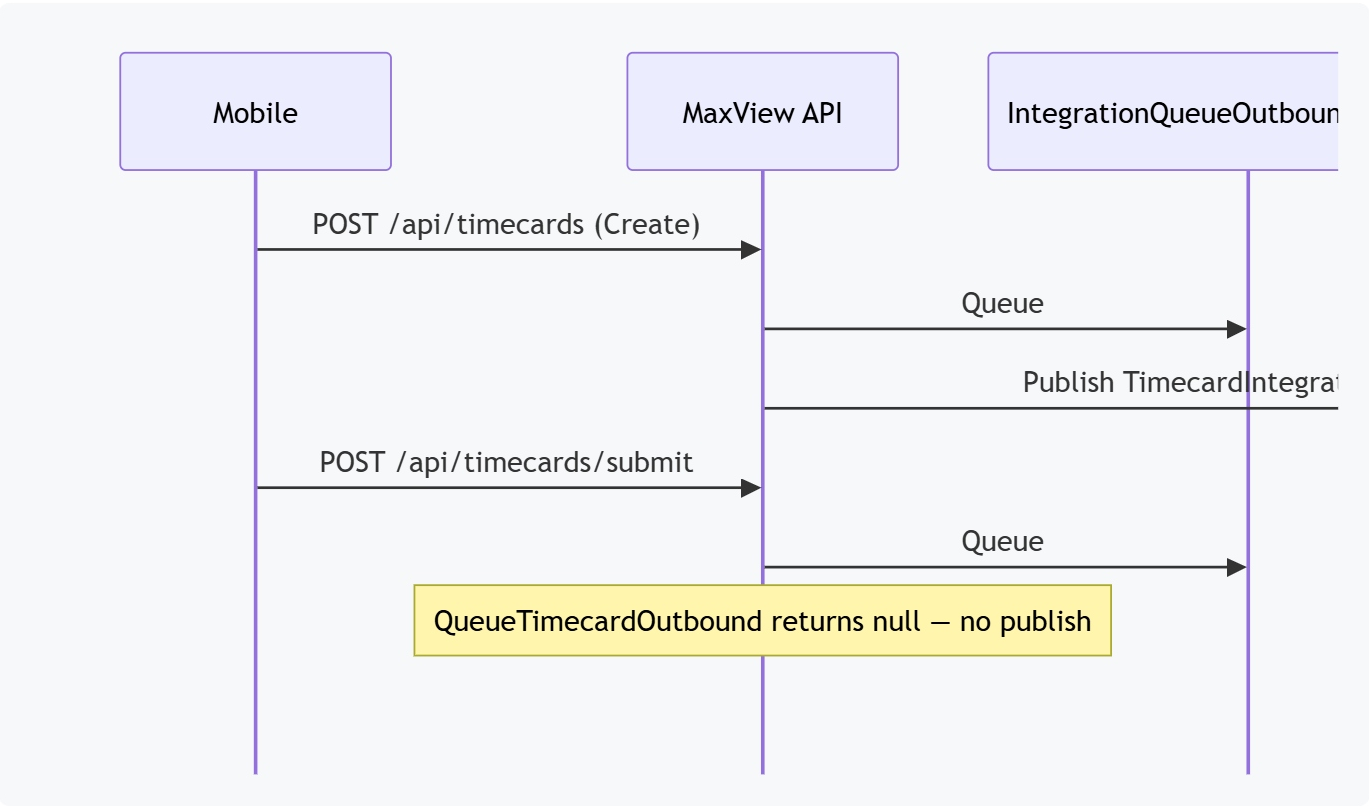
Reuse the existing `IntegrationQueueOutboundTimecards` table as a per-timecard work queue. Only one integration message is **published** at a time per timecard; additional requests **enqueue** a row and defer publish until the current queue item reaches a terminal state

enqueue a row and defer publish until the current queue item reaches a terminal state.

Design principles

- 1. **Enqueue, don't cancel** — when an active queue already exists, still insert the new queue row but return `null` from `QueueTimecardOutbound` so `IntegrateTimecard` skips publishing.
- 2. **Drain on completion** — when a queue item finishes (success, failure, or finished without status integration), advance to the oldest waiting queue for that timecard.
- 3. **FIFO ordering** — completion handlers mark/process queues in `IntegQueueOutTimeId` ascending order (oldest first).

End-to-end flow



Queue lifecycle states

Terminal handler	Trigger	Queue fi
<code>MarkTimecardIntegratedSuccessfully</code>	<code>UpdateTimecardPostQueueRequest</code> after full RC status integration	<code>IsProcess</code> <code>IsSuccess</code>
<code>MarkTimecardIntegratedFailed</code>	<code>/api/timecards/integration/exception</code> after <code>LogIntegrationException</code>	<code>IsProcess</code> <code>HasError=</code>
<code>MarkTimecardIntegrationFinished</code>	<code>UpdateFinishedPostQueueRequest</code> when integration ends without success path	<code>IsProcess</code> only (no

Implementation

1. QueueTimecardOutbound — enqueue instead of cancel

src/Maxim.TimeClock.Infrastructure/.DataAccess/TimecardOperation.cs

- Detect whether an active queue exists (`!IsProcessed && !IsCancelled`).
- Always allow inserting a new queue row when business rules pass (`canCreateQueue`).
- If an active queue existed at call time → **return null** (waiting queue created, publish deferred).
- If no active queue → return the new queue (caller publishes immediately).
- Removed: cancel-previous-queue loop and the `canCreateQueue` guard that blocked a second row while one was active.

2. IntegrateTimecard — publish only when queue is returned

- Calls `QueueTimecardOutbound` ; when result is non-null, delegates to `PublishQueuedTimecardOutboundAsync` (extracted publish/staging/validation logic).
- When result is `null` , integration is queued but no broker message is sent.

3. ContinueNextTimecardOutbound — drain oldest waiting queue

Private helper that:

1. Finds the oldest pending queue for the timecard (`OrderBy IntegQueueOutTimeId`).
2. Refreshes export info / RC external id.
3. Calls `PublishQueuedTimecardOutboundAsync` for that queue id.

Invoked from all three terminal handlers below.

4. Completion handlers (FIFO + drain)

Method	Change
<code>MarkTimecardIntegratedSuccessfully</code>	Order by ascending <code>IntegQueueOutTimeId</code> (was descending); call <code>ContinueNextTimecardOutbound</code> after marking
<code>MarkTimecardIntegratedFailed</code>	Call <code>ContinueNextTimecardOutbound</code> after marking
<code>MarkTimecardIntegrationFinished</code>	Mark <code>IsProcessed=true</code> , <code>ProcessedOn=now</code> ; leave

5. UpdateFinishedPostQueueRequest — finished-without-success callback

New MassTransit contract (TimesheetId , QueueId) published from TimecardIntegrationService.IntegrateTimesheet when:

- The outer validation block succeeds and line entries are upserted, but the inner date/status condition (pay-period gate) is **not** met, **or**
- The method reaches the end without an early success return.

Success path still publishes UpdateTimecardPostQueueRequest and returns early.

Consumer: UpdateFinishedPostQueueConsumer → MarkTimecardIntegrationFinished(QueueId) .

Does **not** run post-success side effects (QueueTimecardProcessingAction , QueueIntegratePdfAttachment) — those remain on the success consumer only.

6. MassTransit wiring

src/Maxim.TimeClock.Web/Program.cs

- Subscription time-card-finished-post → UpdateFinishedPostQueueConsumer
- AddConsumer<UpdateFinishedPostQueueConsumer>()

Files changed

File	
src/Maxim.TimeClock.Infrastructure/DataAccess/TimecardOperation.cs	FIFO q extrac
src/Maxim.TimeClock.Infrastructure/Interface/ITimecardOperation.cs	MarkT
src/Maxim.TimeClock.Infrastructure/Contracts/UpdateFinishedPostQueueRequest.cs	MV-si
src/Maxim.TimeClock.Infrastructure/Consumers/UpdateFinishedPostQueueConsumer.cs	Finishe
src/Maxim.TimeClock.Web/Program.cs	Subscri registr
integration/.../IntegrationService/TimecardIntegrationService.cs	Publis
integration/.../Contracts/UpdateFinishedPostQueueRequest.cs	Integr
	FIFO

tests/Maxim.TimeClock.UnitTests/DataAccessTest/TimecardOperationTest.cs	FIFO n
integration/.../TimecardIntegrationServiceTest.cs	Deferr

Example cases — expected behavior

Case A — mobile add-then-submit (the bug)

Create → queue #1 created → export msg #1 published.
Submit → queue #2 enqueued → QueueTimecardOutbound returns null → **no second publish**.
RC completes msg #1 → success marks queue #1 → drain publishes msg #2.
Msg #2 sees populated ExternalId s → UPDATE path. **One set of RC rows.** ✓

Case B — two different timecards concurrently

Independent queue chains per TimecardId → no cross-timecard blocking. ✓

Case C — first integration fails (validation / RC error)

LogIntegrationException → MarkTimecardIntegratedFailed → drain publishes waiting queue #2. ✓

Case D — line entries upserted but status integration deferred

Inner pay-period gate fails → UpdateFinishedPostQueueRequest → MarkTimecardIntegrationFinished (processed, not marked successful) → drain. ✓

Case E — three rapid submits on same timecard

Queue #1 publishes; #2 and #3 wait. Each completion drains exactly one next queue in FIFO order. ✓

Trade-offs vs Service Bus sessions

Dimension	App-level FIFO (implemented)	Service Bus sessions (deferred)
Strictness	✓ One publish at a time per timecard	✓ Broker-enforced, multi-pod safe
Migration risk	✓ No broker topology change	⚠ Requires entity delete/recreate

Operator work	✅ Deploy code only	⚠️ Drain queue, recreate subscription
Residual race	⚠️ Tiny check-then-insert window without DB lock	✅ Broker handles concurrency
Head-of-line blocking	⚠️ Failed queue blocks drain until marked terminal	⚠️ Same, per session
Observability	Existing queue + exception tables	Session lock / DLQ metrics

Testing

Unit tests (`TimecardOperationTest`)

- `QueueTimecardOutbound_WhenActiveQueueExists_ReturnsNullAndEnqueuesWaitingQueue`
- `MarkTimecardIntegratedSuccessfully_MarksOldestPendingQueueFirst`
- `MarkTimecardIntegrationFinished_MarksQueueProcessedWithoutSuccessFlag`

Integration host test

- `IntegrateTimesheet_ShouldRunSuccessfullyWhenCurrentTimeIsWithinCurrentWeekendingPayPeriodEndDate` — verifies `UpdateFinishedPostQueueRequest` published when inner status gate is not met.

Manual validation (recommended post-deploy)

Reproduce mobile add-then-submit in MV TEST and confirm exactly one set of RC `TimesheetLineEntry` rows.

Verification builds:

```
dotnet build src/Maxim.TimeClock.Infrastructure/Maxim.TimeClock.Infrastructure.csproj
dotnet build src/Maxim.TimeClock.Web/Maxim.TimeClock.Web.csproj
dotnet build integration/TimeClockIntegration/TimeClockIntegration.sln
dotnet test tests/Maxim.TimeClock.UnitTests/Maxim.TimeClock.UnitTests.csproj --filter "Full"
```

ADO Description block:

```
**Surfaces:** Backend only
**Hosts / solutions to verify:** Main TimeClock (TimeClock.sln), TimeClock Integration (Ti
```

Stale queue recovery (production safety)

After deploy, pre-existing active queue rows with a lost broker message or missing completion callback can block the FIFO chain indefinitely. Two recovery mechanisms address this:

Background sweeper

`StaleTimecardOutboundQueueBackgroundService` runs on a configurable interval and calls `RecoverStaleOutboundQueuesAsync` :

1. For each timecard with active queues, inspect the **FIFO head** (oldest `!IsProcessed & & !IsCancelled`).
2. Compute **last activity** as `max(queue.CreatedOn, staging.DataStagedOn)` when staging exists.
3. If last activity is older than `StaleQueueTimeoutMinutes` (default **20**), cancel the head queue (`IsCancelled=true` , `Comments="Stale queue timeout"`) and call `ContinueNextTimecardOutbound` to publish the next waiting queue.

Configuration (`appsettings.json` → `TimecardOutboundQueue`):

Setting	Default	Purpose
<code>SweeperEnabled</code>	<code>true</code>	Toggle background sweeper
<code>SweepIntervalMinutes</code>	<code>5</code>	How often the sweeper runs
<code>StaleQueueTimeoutMinutes</code>	<code>20</code>	Head queue age before cancel + drain

Admin force integration

`GET /api/integration/integrate-timecard/{timecardId}?force=true` (admin API) calls `IntegrateTimecard(timecardId, forceIntegration: true)` :

- Cancels **all** active unprocessed queues for the timecard (`Comments="Force integration"`).
- Creates a fresh queue and publishes immediately (normal integrate path).

Use force for one-off operator recovery; the sweeper handles ambient stale heads automatically.

Deferred alternative — Azure Service Bus sessions

An earlier spike evaluated broker-level FIFO using `SessionId = TCTimecardId` on publish and `RequiresSession = true` on the integration consumer endpoint. That approach remains

valid as a **future hardening** option if the application-level queue proves insufficient under extreme concurrency or multi-pod edge cases.

Key spike findings (unchanged):

- Standard tier supports sessions; topology is MassTransit-provisioned, not Terraform-managed.
- `requiresSession` is immutable — enabling it requires deleting and recreating the subscription.
- All publishers must set `SessionId` or messages are rejected.

No session changes were made in this implementation.

Open questions / follow-ups

- Monitor queue depth for timecards with many rapid edits; confirm drain keeps pace in prod.
- Optional hardening: transactional or serializable insert in `QueueTimecardOutbound` to close the tiny simultaneous-call race window.
- Revisit Service Bus sessions if duplicate RC rows reappear despite this fix